

Algorithms Course Notes

Stephen Arndt

Instructor: Jason Li

Fall 2025

Contents

1	Graph Algorithms	2
1.1	Triangle Counting and Matrix Multiplication	2
1.2	MST and Union-Find	3
1.3	Shortest Path Algorithms	4
2	Dynamic Programming	4
3	Hashing	5
3.1	Basics of Hashing	5
3.2	Advanced Hashing	6
3.3	Load Balancing	6
4	String Algorithms	7
4.1	Suffix Trees	7
4.2	Constructing a Suffix Tree	7
5	Compression in Linear Algebra	9
5.1	PCA and SVD	9
5.2	Streaming Algorithms	9
5.3	Dimensionality Reduction	10
6	Max-Flow Problems	11
6.1	Max Flow Problem	11
6.2	Min-Cost Flow Problem	11
7	Multiplicative Weights	13
7.1	Hedge Algorithm	13
7.2	Solving LPs via MW	13
7.3	Two-Player Zero-Sum Games	14
8	Convex Optimization	16
8.1	Basics of Convex Optimization	16
8.2	Gradient Descent	16

9	Compression and Encoding	17
9.1	Huffman Encoding	17
9.2	Arithmetic Coding	18
9.3	Many Coding Schemes	19
10	Error-Correcting Codes	19
10.1	Basic Definitions and Facts	19
10.2	Reed-Solomon Codes	20
11	Polynomials in Algorithms	20
11.1	k -WIHF from Polynomials	20
11.2	Perfect Matchings and Polynomials	21

1 Graph Algorithms

1.1 Triangle Counting and Matrix Multiplication

In the triangle counting problem, the input is a graph G , and the output is the number of triangles in G .

- **Alg 1:** Iterate over all triples of vertices (u, v, w) .
- **Alg 2:** Iterate over vertices v and pairs of edges $(v, u), (v, w)$ incident to v .
- **Alg 3:** Iterate over vertices v in increasing degree $d_1 \leq d_2 \leq \dots \leq d_n$ and check for triangles to **later** vertices.
- **Alg 4:** Matrix multiplication. Compute $A \otimes A^2$ where A = adjacency matrix of G .

In the matrix multiplication problem, the input is two $n \times n$ matrices A, B , and the output is AB .

- **Alg 1:** Naive algorithm
 - **Runtime 1:** $O(n^3)$
- **Alg 2:** Strassen's algorithm. Multiply 2×2 symbolic matrices using 7 multiplications. Divide-and-conquer.
 - **Runtime 2:** $O(n^{\log_2 7})$ via $T(n) \leq 7 \cdot T(n/2) + \Omega(n^2)$.

Takeaway: For the triangle counting problem, Alg 1 has runtime $O(n^3)$, Alg 2 has $O(mn)$, and Alg 3 has $O(m^{1.5})$. All of these equal $O(n^3)$. Alg 4 has $O(n^{\log_2 7})$ and currently the **best-known** triangle counting algorithm uses reduction to matrix multiplication.

1.2 MST and Union-Find

In the minimum spanning tree problem, the input is a graph G , and the output is an MST of G .

- **Alg:** Kruskal's Algorithm
 - **Alg:** Greedily add min-weight edge that does not form a cycle.
 - **Correctness:** T is a spanning tree (if not, multiple CC's). Prove by induction that $T^k \subseteq T^*$ for some MST T^* .
 - **Runtime:** Union-Find Data Structure. Vertices = Set, Connected Components = Subsets. n MakeSet, $2m$ Find, $n - 1$ Union.

Definition 1.1 (Union-Find Data Structure). A **union-find** data structure is a data structure which given a set S and disjoint subsets of elements $P_1, P_2, \dots, P_k$, supports the operations MakeSet(u), Find(u), and Union(u, v). The array implementation has $\text{UF}[x] = i$ for element x in subset P_i .

Theorem 1.2. M MakeSets, F finds, and U unions can be implemented in $O(M + F + U \log U)$ time.

Proof. (Charging Scheme) Each MakeSet(e) brings $\log_2 n$ dollars stored with e . During a Union, the **smaller** set pays 1 dollar per element. $\square$

Proof. (Potential Function) Let $\phi(s) = \sum_i n_i \log_2(n/n_i)$ where $n_1, n_2, \dots, n_k =$ number of vertices in each set. $\square$

In a charging scheme, **tokens** pay for future operations. In a potential function, **bank account** pays for future operations. In both cases, **amortized cost** is the **actual cost** plus the **change in money**. The charging scheme / potential function should be designed so that **amortized cost** is low for **every operation**.

We now discuss the **tree-based union find** implementation. Each set is a rooted tree with nodes pointing up to root = element. The element corresponds to an ID (easy using an array or hash function).

- **MakeSet(x):** Set x 's parent to itself.
- **Find(x):** Compress traveled path by pointing traveled nodes to root.
- **Union(x, y):** Point shorter tree's root to taller root.
- Keep track of $\text{rank}(r) =$ height of tree rooted at r . $\text{rank}(\text{node}) =$ dead value.

Lemma 1.3. If $\text{rank}(r) = \ell$ then there are $\geq 2^\ell$ nodes in tree.

Lemma 1.4. There are $\leq n/2^\ell$ nodes of rank ℓ .

Theorem 1.5. M MakeSets, F finds, and U unions can be implemented in $O(F \log^* M)$ time.

Proof. Each Union costs two Finds. Charging Scheme. Bucket non-root nodes into buckets $k = 0, 1, 2, \dots, \lceil \log^* n \rceil$ s.t. bucket k has non-root nodes with rank in $[2 \uparrow (k-1), 2 \uparrow k)$. Each time a node joins a new bucket $[r, 2^r)$, withdraw 2^r green tokens. Total number of green tokens withdrawn $\leq n/2^r \cdot 2^r = n$ per bucket, so $\leq n \lceil \log^* n \rceil$ withdrawn in total.

Upon a Find(x), withdraw $2 + \lceil \log^* n \rceil$ red tokens for the path. Use a red token for when $u \rightarrow v$ is a bucket increment, and green token otherwise.

The charging scheme is valid because a) the number of bucket increments (red tokens paid) is $\leq \lceil \log^* n \rceil$, and b) each time u pays a green token, its parent increases rank, so it pays $\leq 2^r$ green tokens before parent increases bucket. $\square$

1.3 Shortest Path Algorithms

In the single-source shortest path problem, the input is a directed graph $D = (V, A)$, source node s , and edge lengths $\ell : A \rightarrow \mathbb{R}$, and the output is the shortest paths from s to all vertices V .

Dijkstra's algorithm solves this problem for positive edge lengths.

- **Alg:** Maintain $W =$ processed vertices, $d_v =$ tentative distances $\forall v \in V$. On each iteration, add minimizer of $d_v : v \notin W$ to W , and update tentative distances using vertex v .
- **Correctness:** Argue $d_v : v \in W$ are actual shortest path lengths to v .
- **Runtime:** Use a PQ. Standard binary heap is $O(\log n)$ time / operation.

Bellman-Ford algorithm solves this problem for all edge lengths.

- **Alg:** Maintain $d_v^t =$ shortest path to v with $\leq t$ edges. Loop over t , loop over $v \in V$, loop over $(x, v) \in A$, and update $d_v^t = \min\{d_v^t, d_x^{t-1} + \ell(x, v)\}$
- **Negative Cycle Detection:** Check $d_v^n = d_v^{n-1} \forall v \in V$.

In the all-pairs shortest path problem, the input is a directed graph $D = (V, A)$, source node s , and edge lengths $\ell : A \rightarrow \mathbb{R}$, and the output is the shortest paths between all pairs of vertices $u, v \in V$.

Floyd-Warshall algorithm solves this problem.

- **Alg:** Maintain $d_{u,v}^{[k]} =$ shortest u, v path using only internal vertices in $1, 2, \dots, k$. Loop over k , loop over u, v , and update $d_{u,v}^{[k]} = \min\{d_{u,v}^{[k-1]}, d_{u,k}^{[k-1]} + d_{k,v}^{[k-1]}\}$.

2 Dynamic Programming

Dynamic programming is an algorithm paradigm which involves 1) finding an optimal substructure, 2) defining the subproblems, and 3) establishing a recurrence. We now give some examples of problems which can be solved by dynamic programming.

In the **longest increasing subsequence** problem, the input is a list of integers $a_1, a_2, \dots, a_n$, and the output is a longest increasing subsequence in $a_1, a_2, \dots, a_n$. Step 1 is to observe that if LIS is $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ then $a_{i_1}, a_{i_2}, \dots, a_{i_{k-1}}$ is LIS ending at i_{k-1} . Step 2 is to define $\text{LIS}[i] = \text{LIS ending at index } i$. Step 3 is to implement the recurrence.

In the **longest common subsequence** problem, the input is two strings S, T , and the output is the longest string s that is a subsequence of both. Step 1 is to observe that if $s = s_1 s_2 \dots s_k$ is the longest string, and s_k is **not** the last character on one of S, T , then we can recurse. Otherwise, s_k is the last character on S, T , and we can recurse. Step 2 is to define $\text{LCS}[i][j] = \text{LCS of } S[1:i], T[1:j]$. Step 3 is to implement the recurrence.

In the **longest path in tree** problem, the input is a rooted tree T , and the output is the longest path in T . Step 1 is to observe that the longest path between two nodes u, v has some highest node s , and the paths $s \rightarrow u, s \rightarrow v$ are the longest paths from s down to the leaves. Step 2 is to define $L[s] = \text{longest path from } s \text{ down tree to leaf}$ and $L'[s] = \text{longest } u-v \text{ path with highest node } s$. Step 3 is to implement the recurrence.

In the **painting an array** problem, the input is an array of integers, and the output is the minimum number of paint strokes needed to “paint” the array. Step 1 is as follows: WLOG assume the first element is painted with the first stroke. In any sequence of strokes, either a) all elements painted with first stroke are repainted, or b) there exists a first surviving element j . $0, j$ partition the array into disjoint instances. Step 2 is to define $S[i][j] = \text{min number of strokes needed for interval } i, i+1, \dots, j$. Step 3 is to implement the recurrence.

3 Hashing

3.1 Basics of Hashing

In **hashing**, the setting is a large universe of keys U and actual dictionary $S \subseteq U$. The operations to support are $\text{add}(x)$, $\text{query}(x)$, $\text{delete}(x)$. We want a hash function $h : U \rightarrow [M]$ with the following three properties: 1) low collisions meaning $x \neq y \in S$ implies $\Pr[h(x) = h(y)] \approx 1/M$, 2) small memory of M, h , and 3) fast computation of h .

Definition 3.1 (Universal Hash Function). h is a **universal** hash function if $\Pr[h(x) = h(y)] = 1/M$ for all distinct $x, y \in U$.

- **Universal HF 1:** $|U| = 2^L, M = 2$. Pick random L -bit vector a . Let $h(x) = a \cdot x \pmod 2$.
- **Universal HF 2:** $|U| = 2^L, M = 2^d$. Pick random $(d \times L)$ -bit matrix A . Let $h(x) = Ax \pmod 2$.

To resolve collisions, we can use **separate chaining** which involves maintaining linked list at each cell of M , or **open addressing** which has no linked lists, and involves linear probing, quadratic probing, etc.

3.2 Advanced Hashing

We start with a useful lemma about collisions in universal hashing.

Lemma 3.2. *For a universal hash function h , with probability $\geq 1/2$ no location has $\geq \sqrt{2M}$ keys mapped to it.*

Proof. Compute expected number of collisions. Use Markov's Inequality. M total collisions implies no location can have $\geq \sqrt{2M}$ keys. $\square$

This leads to **two-level hashing**, where we maintain a second-level table at each location of size $M_i = \Theta(L_i^2)$. Then total expected number of collisions is $\leq 1/2$, so likely no collisions at all.

Definition 3.3 (k -Wise Independent Hashing). h is a **k -wise independent** hash function if $\Pr[h(x_i) = \alpha_i \ \forall i = 1, 2, \dots, k] = 1/M^k$ for all values $\alpha_1, \alpha_2, \dots, \alpha_k$ and distinct keys $x_1, x_2, \dots, x_k$.

Note that 2-wise independent $\implies$ universal.

- **2-WI HF:** $|U| = 2^L$, $M = 2^d$. Pick random $(d \times L)$ -bit matrix A and random d -bit vector b . Let $h(x) = Ax + b \pmod 2$. (2-wise independence not proven in lecture notes).

3.3 Load Balancing

In load balancing, the input is N jobs and M machines, and the output is an assignment of jobs to machines to minimize the maximum load. The assignment algorithm should not query the current assignment of jobs to machines, otherwise this is trivial. The idea is to assign the jobs using hash function(s).

Lemma 3.4. *If $M = N$ and h is 2-wise independent, max load $= O(\sqrt{N})$ with probability $\geq 1/2$.*

Proof. Let L_i = load of machine i . $L_i = X_{i1} + \dots + X_{iN}$. Then $\text{Var}(L_i) \leq 1$. Use Chebyshev with $k = \sqrt{2N}$ and union bound over the machines. $\square$

Lemma 3.5. *If $M = N$ and h is random, max load $= O(\log n)$ with probability $\geq 1 - 1/N$.*

Lemma 3.6. *If $M = N$ and h_1, h_2 random, then max load $= O(\log \log N)$ w/ probability $\geq 1 - 1/N$, when assigning job j to machine $h_1(j), h_2(j)$ with lower load.*

Proof Sketch. Let α_i = fraction of balls which increment a machine to load i . Then $\alpha_i \approx \alpha_{i-1}^2$ and $\alpha_3 \approx 1/4$, so $i = \log \log N$ gives a probability of $\approx 1/N$. $\square$

4 String Algorithms

4.1 Suffix Trees

Definition 4.1 (Trie). A **trie** is a rooted tree for string storage and retrieval. Given a collection of strings $S_1, S_2, \dots, S_m$ over alphabet Σ , a trie is constructed by iterating over the strings $S_1, S_2, \dots, S_m$, starting at the root each time, and branching at each new character. Each node in the trie stores wordcount = number of words passing through it, and endcount = number of words ending at it.

You can also store a single value wordcount at each node and add an end-of-string character $\$$ to every string.

Definition 4.2 (Suffix Tree). A **suffix tree** is a trie which stores all suffixes of a string T , and compresses nodes a with a single child b into a single node ab .

All leaf nodes correspond to suffixes of the string T and so there are $t = |T|$ leaves. Every internal node has at least 2 children, so there are at most t internal nodes. Thus the number of nodes in the suffix tree is at most $2t$.

We can solve the following problems efficiently using a suffix tree.

- **All Occurrences of Pattern:** Given a string T and pattern P , find all occurrences of P in T . Simply follow the pattern P down the suffix tree, and look at the leaves (suffixes) underneath the node reached.
- **All Occurrences of Multiple Patterns:** Given a string T and a stream of patterns $P_1, P_2, \dots$, find all occurrences of each P_i in T . Same as before. The suffix tree is nice because we are very efficient after pre-processing.
- **Lexicographically First Suffix:** Given a string T , find the lexicographically first suffix in T . Simply follow the lexicographically smallest nodes down the suffix tree until reaching a leaf.
- **Longest Repeated Substring:** Given a string T , find the longest repeated substring in T . Simply find the deepest internal node in the suffix tree.

4.2 Constructing a Suffix Tree

For constructing a suffix tree, we use a suffix array and an LCP array.

Definition 4.3 (Suffix Array). A **suffix array** is an array where $SA[i]$ is the start index of the i 'th lexicographically smallest suffix in a string T .

Thus $SA : \text{ranks} \rightarrow \text{indices}$ and $SA^{-1} : \text{indices} \rightarrow \text{ranks}$.

Definition 4.4 (LCP Array). A **LCP array** is an array where $LCP[i]$ is the length of the longest common prefix between the suffixes starting at $SA[i]$ and $SA[i + 1]$ in a string T .

Thus $LCP : \text{ranks} \rightarrow \text{lengths}$.

We now explain how to construct a suffix tree from the suffix array and LCP array. Start with $T = \text{root}$. We will add the suffixes starting at $SA[0], SA[1], \dots, SA[n-1]$ into T . Suppose we are at the leaf for $SA[i-1]$. To add $SA[i]$, look at $LCP[i-1]$ to determine how far up to move in T . Then branch off and make a leaf for $SA[i]$.

The algorithm has runtime $O(n)$, because we construct T by performing a single DFS of T . Constructing the LCP array and suffix array also takes $O(n)$ time, so the full algorithm takes $O(n)$ time.

4.2.1 Kasai's Algorithm: Constructing LCP Arrays

We now explain **Kasai's algorithm** for constructing an LCP array. Start with $i = 0$, and repeatedly update $i \leftarrow SA^{-1}[SA[i] + 1]$. This gives a list of **ranks**. Start with $k = 0$ which tracks the length of the current LCP. On each new i , decrement k , and then increment k by comparing the suffix at $SA[i]$ to the suffix at $SA[i+1]$, and incrementing k on a character match (thus extending the LCP). Set $LCP[i] = k$.

Lemma 4.5. *If $LCP[i] = k$, then $LCP[SA^{-1}[SA[i] + 1]] \geq k - 1$.*

Proof. If $SA[i]$ and $SA[i+1]$ have a longest common prefix of length k , then $SA[i] + 1$ and $SA[i+1] + 1$ have a longest common prefix of length $k - 1$. Thus $LCP[SA^{-1}[SA[i] + 1]] \geq k - 1$. $\square$

Lemma 4.5 establishes the correctness of the algorithm. For the runtime, note that k decrements n times, and increments from 0 to at most n . Thus the total increase in k is at most $2n$, and so the algorithm is $O(n)$ time.

4.2.2 Skew Algorithm: Constructing Suffix Arrays

For sake of time, I will skip explaining the **skew algorithm** for constructing suffix arrays. At a high-level, the algorithm is able to recurse on $2n/3$ suffixes with $O(n)$ overhead, and thus gets $T(n) = O(n) + T(2n/3) \implies T(n) = O(n)$ time. The algorithm also uses radix sort.

4.2.3 Radix Sort

In the “tuple sorting problem”, the input is a list of n k -tuples $x_i = (x_i^1, x_i^2, \dots, x_i^k)$ where $0 \leq x_i^j \leq m-1$ for all i, j , and the output is a sorted list of the n k -tuples. Radix Sort solves this problem in the following way.

- **Radix Sort Algorithm:** First, make km buckets. Next, sort the tuples into m buckets by x_i^k value. Next, process the buckets in increasing order, treating each bucket as a queue, and re-sort the tuples into m new buckets by x_i^{k-1} value. Repeat for k iterations.

- **Correctness:** Consider two tuples x_i, x_j where $x_i < x_j$. Let ℓ be the first coordinate on which x_i, x_j differ, meaning $x_i^h = x_j^h$ for $1 \leq h \leq \ell - 1$ and $x_i^\ell < x_j^\ell$. Then when ℓ is reached in the sorting, x_i will be placed in an earlier bucket than x_j . Then when each $h \leq \ell - 1$ is reached in the sorting, x_i will always be before x_j in the queue. Thus x_i is before x_j in the final sorted order.
- **Runtime:** The runtime is $O(k \max\{m, n\}) = O(n)$ when $k = O(1)$ and $m = O(n)$. Note that this is better than the general $\Omega(n \log n)$ comparison-based sorting lower bound.

5 Compression in Linear Algebra

5.1 PCA and SVD

In the **closest subspace problem**, the input is n points $x_1, x_2, \dots, x_n \in \mathbb{R}^d$ and an integer $k \leq d$, and the output is the subspace V of dimension k minimizing $\sum_i d(x_i, V)^2$. Principal Component Analysis (PCA) solves this problem exactly.

PCA Algorithm: Let $A = \begin{bmatrix} x_1^T \\ x_2^T \\ \dots \\ x_n^T \end{bmatrix}$. For $i = 1, 2, \dots, k$, let $v_i = \arg \max_{\|v\|_2=1, v \perp V_{i-1}} \|Av\|_2$

and $V_i = \text{span}(v_1, v_2, \dots, v_i)$. Output V_k .

Lemma 5.1. $V_k = \text{span}(v_1, v_2, \dots, v_k)$ minimizes $\sum_i d(x_i, V)^2$ over all dimension k subspaces V .

Proof Idea. Minimizing the sum of the distances to V is equivalent to maximizing the lengths of the projections onto V . For $k = 1$, the length of the projections are given by $\langle x_i, v \rangle$, and thus we want to compute $\max_{\|v\|_2=1} \sum_i \langle x_i, v \rangle^2 = \max_{\|v\|_2=1} \|Av\|_2^2$. I don't quite understand why a greedy algorithm works to construct V_k , but it does. $\square$

Definition 5.2 (Singular Vectors and Values). The **right singular vectors** $v_1, v_2, \dots, v_d \in \mathbb{R}^d$ of a matrix A are the vectors computed by PCA. The **left singular vectors** $u_1, u_2, \dots, u_d \in \mathbb{R}^n$ of a matrix A are $u_i = \frac{Av_i}{\|Av_i\|_2}$ for $i = 1, 2, \dots, d$. The **singular values** $\sigma_1, \sigma_2, \dots, \sigma_d$ of a matrix A are $\sigma_i = \|Av_i\|_2$ for $i = 1, 2, \dots, d$.

Theorem 5.3 (SVD). $A = U\Sigma V^T$ where $U = [u_1 \ u_2 \ \dots \ u_d]$, $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_d)$, and $V = [v_1 \ v_2 \ \dots \ v_d]$.

5.2 Streaming Algorithms

In the **L_2 -norm streaming problem**, the input is a universe U of elements and a stream of “add / delete u_i ” operations, and the output is the L_2 -norm of the frequency vector $\vec{f}$.

Theorem 5.4. For all $\epsilon, \delta > 0$, there exists an algorithm for the L_2 -norm streaming problem s.t. the output is within $1 \pm \epsilon$ factor of OPT with probability at least $1 - \delta$, and the algorithm uses $O(\epsilon^{-2}\delta^{-1})$ space.

Algorithm: Pick $k = \frac{2}{\epsilon^2\delta}$. Generate k 4-wise independent hash functions $h_1, h_2, \dots, h_k$ with outputs ± 1 , and k counters $C_1, C_2, \dots, C_k$ initialized to 0. On each “add i ” operation, update $C_j \leftarrow C_j + h_j(i)$ for all j , and on each “delete i ” operation, update $C_j \leftarrow C_j - h_j(i)$. Return $\mu = \frac{1}{k} \sum_i C_i$.

Lemma 5.5. $\mathbb{E}[C_i^2] = \|\vec{f}\|_2^2$ and $\text{Var}(C_i^2) \leq 2 (\mathbb{E}[C_i^2])^2$ for all $i = 1, 2, \dots, k$.

For the proof, simply write $C = \sum_i f_i h(i)$ for a counter C , and work through the computations. It is helpful to remove indexing the counter and hash function by i for notational purposes.

Proof of Theorem 5.4. Via Lemma 5.5 and Chebyshev’s inequality, $\Pr[|C_i^2 - \mathbb{E}[C_i^2]| \geq \epsilon \mathbb{E}[C_i^2]] \leq \frac{\text{Var}(C_i^2)}{(\epsilon \mathbb{E}[C_i^2])^2} \leq \frac{2}{\epsilon^2}$. Note that when summing independent random variables, the mean is the sum of the means, and the variance is the sum of the variances. We can then apply Chebyshev’s inequality in the same way to $C = \frac{1}{k} \sum_i C_i$, obtaining $\frac{2}{\epsilon^2 k} = \delta$ on the right hand side. $\square$

5.3 Dimensionality Reduction

Lemma 5.6 (Johnson-Lindenstrauss Lemma). Let $x_1, x_2, \dots, x_n \in \mathbb{R}^d$. Then there exists an $A \in \mathbb{R}^{k \times d}$ for $k = O(\epsilon^{-2} \log n)$ s.t.

$$\|Ax_i - Ax_j\|_2^2 \in (1 \pm \epsilon) \|x_i - x_j\|_2^2 \quad \forall i, j \in [n]$$

In particular, A consists of independent entries $\frac{N(0,1)}{\sqrt{k}}$.

Lemma 5.6 allows us to compress points in $\mathbb{R}^d$ to $\mathbb{R}^k$ while approximately preserving pairwise distances. Further, the construction of A is simple, fast, and data-oblivious!

Definition 5.7 (Gaussian RVs). The **gaussian random variable** $N(\mu, \sigma^2)$ has probability density function

$$f(x) = \frac{1}{\sqrt{2\pi\sigma}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Fact 5.8 (Scaling and Addition). Gaussian random variables are closed under **scaling** and **addition**. In particular, $cN(\mu, \sigma^2) = N(c\mu, c^2\sigma^2)$ and $N(\mu_1, \sigma_1^2) + N(\mu_2, \sigma_2^2) = N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$.

Lemma 5.9. $\mathbb{E}[\|Ax\|_2^2] = 1$ for all unit vectors $x \in \mathbb{R}^d$.

Proof. $(Ax)_i = a_i^T x = a_{i1}x_1 + a_{i2}x_2 + \dots + a_{in}x_n$ is a linear combination of gaussian random variables $\frac{N(0,1)}{\sqrt{k}}$. Via Fact 5.8, $(Ax)_i = N(0, \frac{1}{k} \sum_i x_i^2) = N(0, \frac{1}{k})$. Thus $\mathbb{E}[(Ax)_i^2] = \frac{1}{k}$, giving $\mathbb{E}[\|Ax\|_2^2] = 1$. $\square$

Lemma 5.10. For all unit vectors $x \in \mathbb{R}^d$, $\Pr[\|Ax\|_2^2 \notin 1 \pm \epsilon] \leq \delta$ when using $k = O(\epsilon^{-2} \log \delta^{-1})$.

Proof Idea. The proof idea is to treat $N(0, 1)^2$ as a bounded random variable in $[-1, 1]$, and thus $\frac{N(0, 1)^2}{k}$ as a bounded random variable in $[-1/k, 1/k]$. Thus via Hoeffding's Inequality, $\Pr[\|Ax\|_2^2 \notin 1 \pm \epsilon] \leq 2 \exp\left(-\frac{2\epsilon^2}{k(2/k)^2}\right) = 2 \exp(-k\epsilon^2/2)$. For $k = \Theta(\epsilon^{-2} \log \delta^{-1})$, this is $e^{-\Omega(\log \delta^{-1})} = \text{poly}(\delta)$. $\square$

Proof of Lemma 5.6. Pick $\delta = 1/n^3$ and apply Lemma 5.10. There are $O(n^2)$ unit vectors $\frac{x_i - x_j}{\|x_i - x_j\|}$ for all $i, j \in [n]$, which induces $O(n^2)$ bad events of high deviation. Union bound over the bad events to achieve the desired result. $\square$

Note: In the L_2 -norm streaming problem, the vector of counters is $\vec{C} = H^T \vec{f}$. H is essentially a random bit matrix, but not quite, as we only have 4-wise independence on the hash functions. This gives some explanation for why $k = O(\epsilon^{-2} \delta^{-1})$ hash functions are required, while Lemma 5.10 would require only $k = O(\epsilon^{-2} \log \delta^{-1})$ hash functions under full independence.

6 Max-Flow Problems

6.1 Max Flow Problem

In the **max flow problem**, the input is a directed graph $G = (V, A)$, a source s , a sink t , and capacities $c : A \rightarrow \mathbb{R}^+$, and the output is a max flow $f : A \rightarrow \mathbb{R}^+$. A flow must satisfy the capacity and flow conservation constraints, and the objective is $\max \sum_v f(s, v)$.

The **Ford-Fulkerson algorithm** solves the max flow problem exactly. Initially $f = 0$ everywhere. Always maintain that f is unidirectional, i.e. f does not send flow across both (u, v) and (v, u) . Define the residual flow graph G_f where $c_f(u, v) = c(u, v) - f(u, v)$ and $c_f(v, u) = c(v, u) + f(u, v)$. While G_f contains an $s - t$ path of minimum capacity $m > 0$, add a flow of size m along that path, and update f .

For correctness, we will find an (s, t) -cut (S, T) which is saturated. Let S = set of nodes reachable from s in the final residual graph G_f . For all arcs from S to T , we have $c_f(u, v) = 0$ and so $c(u, v) = f(u, v)$. For all arcs from T to S , we have $f(v, u) = 0$ because otherwise an arc in G_f from u to v would form. Thus the (s, t) -cut (S, T) is saturated.

The runtime is $O(EF)$ where F is the max flow value, assuming capacities are integers.

6.2 Min-Cost Flow Problem

In the **min-cost flow problem**, the input is a directed graph G , a source s , a sink t , capacities $c : A \rightarrow \mathbb{R}^+$, and costs $\$: A \rightarrow \mathbb{R}^+$, and the output is a max flow $f : A \rightarrow \mathbb{R}^+$ which has minimum cost. For simplicity, we also assume that G has **unidirectional** capacities, i.e. $c(u, v) > 0 \implies c(v, u) = 0$.

The **cycle-canceling algorithm** solves this problem exactly. First, compute a max flow f . Build the residual graph G_f where $\$_f(u, v) = \(u, v) and $\$_f(v, u) = -\(u, v) for edges (u, v) where f has nonzero flow, and $\$_f(u, v) = \(u, v) for edges (u, v) where f has zero flow. Find a negative-cost cycle in G_f and add flow along that cycle. Update f and repeat.

Note: Because G has unidirectional capacities, the costs on edges in G_f are the same regardless of the flow f . $\$_f(u, v)$ is either always positive (and equal to $\$(u, v)$) or always negative (and equal to $-\$(v, u)$).

Definition 6.1. A flow f is **cost-optimal** if it has minimum cost among all flows of equal value.

Lemma 6.2. A flow f is cost-optimal iff there is no negative cost cycle in G_f .

Proof. The forwards implication is easy. For the backwards implication, suppose f is not cost-optimal. Let f^* be a cost-optimal flow and consider the flow $f^* - f$, where negative flow is put on a reversed edge. This is a circulation flow (a flow of value 0) in G_f with negative cost. All circulation flows can be decomposed into cycles, and so some cycle in G_f has negative cost, a contradiction. $\square$

The runtime is $O(EF + ECD \cdot EV) = O(E^2VCD)$ where $C = \max_e c(e)$, $D = \max_e \$(e)$ because the initial max-flow computation takes time $O(EF)$, the max possible cost is ECD , and each iteration decrementing the cost takes time $O(EV)$ using the Bellman-Ford algorithm.

The **cheap augmenting paths algorithm** also solves this problem exactly. Run Ford-Fulkerson and always update along the cheapest augmenting $s-t$ path in G_f .

Lemma 6.3. If G_f has no negative-cost cycles, and f' is the updated flow of f along a cheapest augmenting path, then $G_{f'}$ has no negative-cost cycles.

Proof. Let $d(v) = s - v$ distance in G_f and $d'(v) = s - v$ distance in $G_{f'}$. We will show that $d'(v) \geq d(v)$ for all $v \in V$, and thus $G_{f'}$ has no negative-cost cycles. AFSOC not, i.e. there exists a $w \in V$ s.t. $d'(w) < d(w)$. Let P be the shortest $s - w$ path in $G_{f'}$. Go backwards from w on P , to find the latest pair of consecutive vertices u, v on P where $d'(u) \geq d(u)$ and $d'(v) < d(v)$. We know $d'(v) = d'(u) + \$_{f'}(u, v)$. Thus $(u, v) \notin G_f$, because otherwise $d(v) \leq d(u) + \$_f(u, v)$, a contradiction. Thus the cheapest augmenting path in G_f must use (v, u) . But then $d(u) = d(v) + \$_f(v, u)$ giving $d(v) = d(u) + \$_f(u, v)$, a contradiction. Thus $\forall w \in V$ we have $d'(w) \geq d(w)$. $\square$

Thus the cycle-canceling algorithm maintains a flow f which is **maximum** and decreases the cost, while the cheapest augmenting paths algorithm maintains a flow which is **cost-optimal** and increases the flow.

LPs and MFMC: If max-flow is the primal LP, then min-cut is the dual LP. Further, if the capacities are integral, then both LPs are integral. Thus the Max-Flow Min-Cut Theorem can be viewed as a consequence of Strong Duality.

7 Multiplicative Weights

7.1 Hedge Algorithm

In the **expert predictions setting**, on each day, N experts make a prediction about a binary outcome, you see these predictions and make your own prediction, and then the binary outcome is revealed. Your goal is to minimize your **regret**, which is defined as the difference between the best expert's score and your score. This setting can be generalized by giving each expert a **gain** $g_i^{(t)}$ on step t , which is 0, 1 (based on incorrect, correct) in the binary outcome setting, and attempting to minimize the difference between the best expert's gain and your gain. In the generalized setting, you don't control your gains (otherwise this is trivial), but instead you can pick an expert i to match with before their gain is revealed.

The **hedge algorithm** is a simple and effective algorithm. Let $\epsilon > 0$. Initialize weights $w_i^{(1)} \leftarrow 1$ for all $i \in [N]$. On each day t , let $W_t = \sum_j w_j(t)$ be the total weight of the experts, and define probability vector $p_i^{(t)} \leftarrow w_i^{(t)}/W_t$. Then pick expert i with probability $p_i^{(t)}$ and match their prediction. After the outcome is revealed, update $w_i^{(t+1)} \leftarrow w_i^{(t)} \exp(\epsilon g_i^{(t)})$ where $g_i^{(t)} = 0, 1$ is the gain or score of expert i on step t .

Theorem 7.1. *If $g^{(t)} \in [-1, 1]^N$ for all t , then using the hedge algorithm,*

$$\sum_{t=1}^T \langle p^{(t)}, g^{(t)} \rangle \geq \sum_{t=1}^T g_i^{(t)} - \epsilon T - \frac{\ln N}{\epsilon} \quad \forall i \in [N]$$

Thus your expected regret is at most $\epsilon T + \frac{\ln N}{\epsilon}$ after T rounds. Setting $T = \frac{\ln N}{\epsilon^2}$ gives expected regret $\leq 2\epsilon T$.

Proof Idea. Potential function argument. Use $\Phi^{(t)} = \sum_j w_j(t)$. Upper bound $\Phi^{(t+1)}$ in terms of $\Phi^{(t)}$, giving an upper bound on $\Phi^{(T)}$, giving an upper bound on w_i^T for all experts $i \in [N]$, which produces the inequality. $\square$

Remark: If $g^{(t)} \in [-\rho, \rho]^N$ for all t , you can pick $T = O\left(-\frac{\rho^2 \ln(N)}{\epsilon^2}\right)$, and still obtain expected regret $\leq 2\epsilon T$. ρ is sometimes called the width.

7.2 Solving LPs via MW

Let $P = \max\{c^T x : Ax \leq b, x \geq 0\}$.

Algorithm: Play as the adversary to the hedge algorithm. Let $m =$ number of constraints. Each constraint is an expert. On each round t , where hedge algorithm has probability vector $p^{(t)}$, solve $\max\{c^T x : (p^{(t)})^T Ax \leq (p^{(t)})^T b, x \geq 0\}$. Let optimal solution be $x^{(t)}$. Play $g^{(t)} = Ax^{(t)} - b$. Output $\bar{x} = \frac{1}{T} \sum_{t=1}^T x^{(t)}$.

Intuition for Algorithm: Let's take a look at the gain vector $g^{(t)}$. If $g_i^{(t)}$ is high, then constraint i is heavily violated, and hedge algorithm will respond

correctly by increasing the weight of constraint i for the next round. Similarly if $g_i^{(t)}$ is low, then constraint i has a lot of slack, and hedge algorithm will respond correctly by decreasing the weight of constraint i for the next round.

For simplicity of analysis, we will assume $g^{(t)} \in [-\rho, \rho]$ for all t for some $\rho > 0$.

Theorem 7.2. *For $\epsilon > 0$, after $T = O\left(\frac{\rho^2 \ln N}{\epsilon^2}\right)$ rounds, $\bar{x}$ is **superoptimal** and ϵ -**approximately feasible**, i.e. $c^T \bar{x} \geq OPT$ and $A\bar{x} \leq b + \epsilon \mathbf{1}, \bar{x} \geq 0$.*

Proof. Hedge algorithm guarantees that

$$\sum_{t=1}^T \langle p^{(t)}, g^{(t)} \rangle \geq \sum_{t=1}^T g_i^{(t)} - 2\epsilon T \quad \forall i \in [m]$$

for the chosen value of T . Note that $\langle p^{(t)}, g^{(t)} \rangle \leq 0$ for all t , and $g_i^{(t)} = (Ax^{(t)})_i - b_i$ is the constraint violation or slack for constraint i on step t . Simplifying shows the ϵ -approximate feasibility. Further, $x^{(t)}$ is superoptimal for all t , so $\bar{x}$ is superoptimal. $\square$

Remark: This procedure can be generalized in the following way: instead of preserving the $x \geq 0$ constraint on every round, we can preserve a set of K “simple” constraints on every round, and take linear combinations of the remaining “complicated” constraints. We can then guarantee superoptimality, feasibility on the simple constraints, and approximate feasibility on the complicated constraints.

7.3 Two-Player Zero-Sum Games

Definition 7.3. A **two-player zero sum game** with payoff matrix $M \in \mathbb{R}^{m \times n}$ is the following game. There is a row player (the maximizer) and a column player (the minimizer). The row player picks a row $i \in [m]$ and the column player picks a column $j \in [n]$ (both plays occur simultaneously). The output is M_{ij} .

Given a row player strategy $p \in \Delta_m$ and column player strategy $q \in \Delta_n$, the expected payoff is $p^T M q$. Let's now study the optimal column player and row player strategies. Define $C(p) = \min_{q \in \Delta_n} p^T M q = \min_{j \in [n]} p^T M e_j$ as the optimal column player payoff given knowledge of $p \in \Delta_m$. Define $R(q) = \max_{p \in \Delta_m} p^T M q = \max_{i \in [m]} e_i^T M q$ as the optimal row player payoff given knowledge of $q \in \Delta_n$.

Lemma 7.4 (Weak Minimax Theorem). $\forall p \in \Delta_m$ and $\forall q \in \Delta_n$, $C(p) \leq R(q)$.

The proof is direct. The intuition is that there is a **second-player advantage** to this game if the turns are taken sequentially.

Theorem 7.5 (Von Neumann Minimax Theorem). $\max_{p \in \Delta_m} C(p) = \min_{q \in \Delta_n} R(q)$.

Thus there exists p, q s.t. there is no actual second-player advantage!

Proof. We will prove that $\forall \epsilon > 0$ there exist $\bar{p}, \bar{q}$ s.t. $C(\bar{p}) \geq R(\bar{q}) - \epsilon$. $C(p)$ and $R(q)$ are continuous functions on compact sets, and thus by Extreme Value Theorem, they attain maxima and minima. Thus the claim proves that $\max C(p) = \min R(q)$.

To prove the claim, we use the Hedge algorithm. Scale M down until $M \in [-1, 1]^{m \times n}$. The experts are the m rows. On round t , the gain vector is $g^{(t)} = Mq^{(t)}$ where $q^{(t)} = \arg \min_{q \in \Delta_n} (p^{(t)})^T Mq$. After T rounds, pick $\bar{p} = \frac{1}{T} \sum_{t=1}^T p^{(t)}$ and $\bar{q} = \frac{1}{T} \sum_{t=1}^T q^{(t)}$.

Conceptually the probability vector is the row player strategy, and the gain vector is the column of payoffs under an optimal column player strategy. Thus the expected gain per round is the expected payoff. $\bar{p}, \bar{q}$ are the average row, column player strategies after many rounds.

Hedge algorithm guarantees that after $T = O\left(\frac{\ln m}{\epsilon^2}\right)$ rounds, we have

$$\begin{aligned} \sum_{t=1}^T \langle p^{(t)}, g^{(t)} \rangle &\geq \max_{i \in [n]} \sum_{t=1}^T \langle e_i, g^{(t)} \rangle - 2\epsilon T \\ \frac{1}{T} \sum_{t=1}^T \langle p^{(t)}, g^{(t)} \rangle &\geq \max_{i \in [n]} \frac{1}{T} \sum_{t=1}^T \langle e_i, g^{(t)} \rangle - 2\epsilon \end{aligned}$$

The LHS simplifies to $\leq C(\bar{p})$ and the RHS simplifies to $R(\bar{q}) - 2\epsilon$ after substituting $g^{(t)} = Mq^{(t)}$. $\square$

We can actually prove **strong duality** using the minimax theorem! Let's first do the case where $b = c = \mathbf{1}$. The minimax theorem says that

$$\max_{p \in \Delta_m} \min_{j \in [n]} p^T M e_j = \min_{q \in \Delta_n} \max_{i \in [m]} e_i^T M q$$

The LHS can be simplified as

$$\begin{aligned} \max_{p \in \Delta_m} \min_{j \in [n]} p^T M e_j &= \max\{t : M^T p \geq t \mathbf{1}, \mathbf{1}^T p \leq 1, p \geq 0\} \\ &= \max\{t : M^T x \geq \mathbf{1}, \mathbf{1}^T x \leq 1/t, x \geq 0\} \\ &= [\min\{\mathbf{1}^T x : M^T x \geq \mathbf{1}, x \geq 0\}]^{-1} \end{aligned}$$

Similarly, the RHS can be simplified as

$$\min_{q \in \Delta_n} \max_{i \in [m]} e_i^T M q = [\max\{\mathbf{1}^T y : M y \leq \mathbf{1}, y \geq 0\}]^{-1}$$

Thus the minimax theorem shows the primal LP and dual LP have strong duality! I believe that for general b, c , you can transform into $b = c = \mathbf{1}$ by transforming the payoff matrix as well.

8 Convex Optimization

8.1 Basics of Convex Optimization

Fact 8.1. A differentiable function $f : K \rightarrow \mathbb{R}$ is convex iff $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle$ for all $x, y \in K$.

Fact 8.2. If $f : K \rightarrow \mathbb{R}$ is convex and differentiable and $\nabla f(x) = 0$, then $x = \min_{x \in K} f(x)$.

Definition 8.3. $f : K \rightarrow \mathbb{R}$ is G -**Lipschitz** if $|f(x) - f(y)| \leq G\|x - y\|_2$ for all $x, y \in K$.

Fact 8.4. For a differentiable function $f : K \rightarrow \mathbb{R}$, f is G -**Lipschitz** iff $\|\nabla f(x)\|_2 \leq G$ for all $x \in K$.

8.2 Gradient Descent

Algorithm: Initialize $x_0 \in K$. For $t = 0, 1, \dots, T - 1$, update $x_{t+1} \leftarrow x_t - \eta \cdot \nabla f(x_t)$. Return $\bar{x} = \frac{1}{T} \sum_{t=1}^T x_t$.

Theorem 8.5 (Gradient Descent). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be convex, differentiable, and G -Lipschitz. Let $L = \max_{x, y \in K} \|x - y\|_2$. Then for all $x \in K$,

$$\frac{1}{T} \sum_{t=1}^T f(x_t) \leq f(x) + \frac{1}{2}\eta G^2 + \frac{1}{2\eta T} L^2$$

Note that picking $T = \frac{G^2 L^2}{\epsilon^2}$ and $\eta = \frac{1}{G\sqrt{T}}$ gives $f(\bar{x}) \leq f(x^*) + \epsilon$. It's important that f is convex, because we use the fact that $f(\bar{x}) \leq LHS$.

Proof Idea. Define potential function $\Phi_t = \frac{\|x_t - x^*\|_2^2}{2\eta}$. As a lemma, establish $f(x_t) + (\Phi_{t+1} - \Phi_t) \leq f(x^*) + \frac{1}{2}\eta G^2$. Sum over $t = 1, 2, \dots, T$ to get the desired result. $\square$

In **online gradient descent**, the algorithm plays $x_t \in K$, and the adversary plays a convex function f_t . The same proof gives $\frac{1}{T} \sum_{t=1}^T f_t(x_t) \leq \frac{1}{T} \sum_{t=1}^T f_t(x) + \frac{1}{2}\eta G^2 + \frac{1}{2\eta T} L^2$. We can think about $x \in K$ as “experts”, and the theorem states that online gradient descent can perform well even compared to the best expert.

In **projected gradient descent**, we project x_t back into K after each update step. I believe that essentially the same proof works and you can obtain the same regret bounds.

9 Compression and Encoding

9.1 Huffman Encoding

Definition 9.1. The **entropy** of a set $S = \{s_1, s_2, \dots, s_m\}$ with probabilities $p : S \rightarrow [0, 1]$, i.e. a discrete probability distribution, is $H(S) = -\sum_{s \in S} p(s) \log_2(p(s))$.

If probabilities are uniform, $H(S) = \log_2 m$. If $p(s_1) \rightarrow 1$ and $p(s_2), p(s_3), \dots, p(s_m) \rightarrow 0$, then $H(S) \rightarrow 0$.

Definition 9.2. A **code** C for message set S is a mapping $C : S \rightarrow \{0, 1\}^*$.

Note that $\{0, 1\}^*$ is the set of finite bitstrings.

Definition 9.3. A **prefix code** C for message set S is a code s.t. no codeword is a prefix of another codeword.

Prefix codes are uniquely decodable, i.e. a string of codewords generated by a prefix code is uniquely decodable. A prefix code can be viewed as a binary tree, where the leaves are the codewords.

Definition 9.4. The **average length** of a code C is $\ell(C) = \mathbb{E}_{s \sim S}[|C(s)|]$.

Theorem 9.5 (Kraft-McMillan). *If $C : S \rightarrow \{0, 1\}^*$ is a uniquely decodable code, then $\sum_{s \in S} 2^{-|C(s)|} \leq 1$. Conversely, for any integers $\ell_1, \ell_2, \dots, \ell_m$ s.t. $\sum_{i=1}^m 2^{-\ell_i} \leq 1$, there exists a prefix code C with lengths $\ell_1, \ell_2, \dots, \ell_m$.*

Proof ($\implies$). We will just prove this for prefix codes. Let $\ell_{\max}$ be the maximum length of a codeword. Consider the set of $2^{\ell_{\max}}$ bitstrings of length $\ell_{\max}$. Each codeword of length $\ell \leq \ell_{\max}$ eliminates $2^{\ell_{\max}-\ell}$ of these bitstrings from being used as a codeword. Further, the codewords eliminate disjoint sets of these bitstrings. We need $\sum 2^{\ell_{\max}-\ell_i} \leq 2^{\ell_{\max}}$, giving the desired result. $\square$

Proof ($\impliedby$). WLOG $\ell_1 \leq \ell_2 \leq \dots \leq \ell_m$. We can build a complete binary tree T with 2^{ℓ_m} leaves, and build a prefix code by adding codewords and pruning the tree. $\square$

Fact 9.6 (Jensen's Inequality). *Let $f : K \rightarrow \mathbb{R}$ be a convex function, $x_1, x_2, \dots, x_m \in K$, $p_1 + p_2 + \dots + p_m = 1$, and $p_i \geq 0$ for all i . Then*

$$f\left(\sum_{i=1}^m p_i x_i\right) \leq \sum_{i=1}^m p_i f(x_i)$$

The inequality is reversed for concave functions. Jensen's Inequality can easily be proved from iteratively using the definition of convex functions.

Lemma 9.7. *For any uniquely decodable code C , $\ell(C) \geq H(S)$.*

Proof. Directly compute $\ell(C) - H(S) = \sum p(s) \log_2(\text{stuff})$, apply Jensen's Inequality using the concave function $f(x) = \log_2 x$, and apply the Kraft-McMillan inequality. $\square$

Lemma 9.8. *There exists a prefix code C with $\ell(C) \leq H(S) + 1$.*

Proof. Assign each s a codeword of length $\lceil \log \left(\frac{1}{p(s)} \right) \rceil$. Prove the Kraft-McMillan Inequality holds, so you can generate a prefix code C with the desired lengths, and prove that $\ell(C) \leq H(S) + 1$. $\square$

Huffman Encoding Algorithm: Initialize a graph G with vertices $= S$, no edges, and $w(s) = p(s)$. Iteratively select two trees with the lowest weights, and combine them under a common parent with weight = sum of weights. View final tree T as a prefix code C .

Lemma 9.9. *Huffman Encoding Algorithm generates an optimal prefix code C^* , i.e. for all prefix codes C , $\ell(C^*) \leq \ell(C)$.*

Proof. Induction on m . Get an optimal code C^* on S . Let x, y be lowest-weight strings in S . Argue that WLOG C^* puts x, y as siblings on its lowest level via an exchange argument. Consider $S' = S \setminus \{x, y\} \cup \{z\}$. We can define four codes, namely C^* for S , $C^{*'} for S' , C for S (from Huffman), and C' for S' (from Huffman). Ultimately you want to show $\ell(C) \leq \ell(C^*)$. Expanding out in terms of S' , and using the optimality of Huffman on S' from the induction, gives the desired result. $\square$$

9.2 Arithmetic Coding

We will now define a code $C : S^n \rightarrow [0, 1]$ that can encode sequences of strings of length n . The code will require on average $\leq n \cdot H(S) + 2$ bits to send a length n message, as opposed to the $\leq n \cdot H(S) + n$ bits guaranteed by optimal prefix codes.

Arithmetic Coding Algorithm: Given probabilities $p_1, p_2, \dots, p_m$, split $[0, 1]$ into $[0, p_1), [p_1, p_2), \dots, [p_m, 1)$. Further split the intervals in n total rounds, always splitting proportional to the probabilities $p_1, p_2, \dots, p_m$. Assign a codeword to each interval s.t. $[0.\text{codeword}0, 0.\text{codeword}1) \subseteq \text{interval}$.

Correctness: The code C is a prefix code based on the disjointness of the intervals. It remains to show how to pick the codewords efficiently (i.e. a small size).

Lemma 9.10. *The interval $[a, a + b)$ can be represented by $a + b/2$ truncated to $\lceil -\log_2 b \rceil + 1$ bits.*

Proof. Let $x \in [0, 1]$ be the value of the codeword. Argue $a \leq x$. Then argue $0.\text{codeword}1 = x + 2^{-|\text{codeword}|} = x + 2^{-\lceil -\log_2 b \rceil - 1} \leq x + b/2 \leq (a + b/2) + b/2$. $\square$

Thus for the length n message $m_1 m_2 \dots m_n$, we have $b = p(m_1) \cdot p(m_2) \cdot \dots \cdot p(m_n)$, and the length of the codeword is $\lceil \sum \log_2(1/p_i) \rceil + 1$. This leads to expected length $\leq n \cdot H(S) + 2$.

9.3 Many Coding Schemes

In **run-length encoding**, a message *abbcccb* is encoded to $(a, 1)(b, 2)(c, 3)(b, 1)$. In **move-to-front encoding**, we start with a total order over the alphabet $[a, b, c, d, \dots]$. Then to encode character *c*, we put down a 3 and move *c* to the front of the order, giving $[c, a, b, d, \dots]$. The hope is that characters with similar positions in the order will appear in similar places, so the integers will be small. In **residual coding**, we have an algorithm for guessing the next value based on the current context, and then store the difference between guess value and actual value.

In the **Burrows-Wheeler Transform**, we sort all cyclical shifts of message\$ lexicographically, list them out, and then take the last column of the formed matrix. A few important features are a) the original string can be recovered from the last column, b) this is a useful preprocessing step for run-length encoding or move-to-front encoding, and c) this can be implemented efficiently using a suffix array. For sake of time, I won't explain why these properties hold.

In **linear transform coding**, the goal is to pick linear basis functions which span the space of messages, decorrelate the data and get low coefficients for many terms, then set the low coefficients to 0. This is a useful preprocessing technique for lossy compression of images (lossy is okay because we only care about visual perception).

Quantization is a general term describing compression algorithms which compress a large space of messages into a small space of codewords (ex. by dropping the least significant bit), i.e. **lossy compression** algorithms.

10 Error-Correcting Codes

In the **noisy codes setting**, a message *m* is encoded to codeword *c*, passed through a noisy channel producing codeword *c'*, and then decoded back to message *m* or output ERROR. An **error** is a changed coordinate of *c* to a valid character, and an **erasure** is a changed coordinate of *c* to ?. Errors require both detection and correction, while erasures require only correction.

10.1 Basic Definitions and Facts

Definition 10.1. A **block code** maps length *k* messages to length *n* codewords where $k \leq n$.

Definition 10.2. A **systematic code** has the message appear directly in the codeword.

Definition 10.3. The **hamming distance** $\Delta(x, y)$ between two vectors $x, y \in \Sigma^n$ is the number of character differences between *x*, *y*.

Definition 10.4. The **minimum distance** *d* of a code *C* is $d = \min\{\Delta(x, y) : x, y \in C, x \neq y\}$.

Minimum distance imposes a constraint on error correction. For example, we need $d \geq 3$ to correct a one-bit error.

We now establish an important fact about polynomials over a finite field $\mathbb{F}$.

Fact 10.5. *Every degree $k - 1$ polynomial over a finite field $\mathbb{F}$ is uniquely determined by k distinct samples of the polynomial.*

Proof. Let $h(x) = c_0 + c_1x + \dots + c_{k-1}x^{k-1}$ be a degree $k - 1$ polynomial over finite field $\mathbb{F}$. Let $h(x_i) = y_i$ for $i = 1, 2, \dots, k$ be k distinct samples of

the polynomial. Thus $Vc = y$ where $V = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^{k-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{k-1} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_k & x_k^2 & \dots & x_k^{k-1} \end{bmatrix}$ is the

Vandermonde matrix of the x_i values, which is known to be a full-rank square matrix for any distinct x_i values. Thus $c = V^{-1}y$ is the only solution, and so $h(x)$ is the only degree $k - 1$ polynomial interpolating the samples. $\square$

Another proof of Fact 10.5 is to get two polynomials g, h which go through the k data points, and define $f = g - h$. Then f has degree at most $k - 1$ and k zeros, and thus $f = 0$, so $g = h$.

10.2 Reed-Solomon Codes

Definition 10.6. The **Reed-Solomon Code** for converting length k messages to length n codewords for $k \leq n$ involves a finite field $\mathbb{F}_n$, and maps the message $(c_0, c_1, \dots, c_{k-1})$ to the codeword $(p(0), p(1), \dots, p(n-1))$, where $p(x) = c_0 + c_1x + \dots + c_{k-1}x^{k-1}$.

Lemma 10.7. *Given any $n - k$ erasures of the codeword, we can uniquely decode the message.*

This follows immediately from Fact 10.5. Note that Reed-Solomon Code = Vandermonde Matrix of $0, 1, \dots, n-1 \times$ Coefficient Vector. Thus Reed-Solomon Code = Constant Matrix $\times$ Message. It is a linear operator!

11 Polynomials in Algorithms

11.1 k -WIHF from Polynomials

Now suppose we sample random coefficients $c_0, c_1, \dots, c_{k-1} \in \mathbb{F}$ from a finite field $\mathbb{F}$ and let $h(x) = c_0 + c_1x + \dots + c_{k-1}x^{k-1}$.

Lemma 11.1. *h is a k -wise independent hash function, i.e. $\Pr[h(i_1) = a_1, h(i_2) = a_2, \dots, h(i_k) = a_k] = \frac{1}{|\mathbb{F}|^k}$ for all $i_1, i_2, \dots, i_k, a_1, a_2, \dots, a_k \in \mathbb{F}$.*

This follows immediately from Fact 10.5. The probability of interpolating the k samples is the probability of getting a specific coefficient vector c .

11.2 Perfect Matchings and Polynomials

In the **polynomial testing problem**, we are given an unknown polynomial p , and we want to determine whether $p = 0$.

Lemma 11.2 (Schwartz-Zippel Lemma). *Let $p(x_1, x_2, \dots, x_n)$ be a nonzero polynomial over field $\mathbb{F}$ and $d = \deg p$. Let $S \subseteq \mathbb{F}$ be arbitrary and pick n random samples $r_1, r_2, \dots, r_n$ of S . Then $\Pr[p(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|S|}$.*

Thus if p has low degree, we can repeatedly generate uniform random samples from $\mathbb{F}$ to test whether $p = 0$.

Application: Determining the existence of a perfect matching in a bipartite graph $G = (A, B, E)$ where $|A| = |B|$.

Definition 11.3. The **Edmonds matrix** $E(G)$ of bipartite graph $G = (A, B, E)$ has $E(G)_{ij} = x_{ij}$ if $(i, j) \in E$, 0 otherwise.

Lemma 11.4. *Let $G = (A, B, E)$ be a bipartite graph. There exists a perfect matching in G if and only if $\det E(G) \neq 0$.*

Proof. $\det E(G) = \sum_{\text{bijections } \sigma: [n] \rightarrow [n]} \text{sign}(\sigma) \prod_{i=1}^n E(G)_{i\sigma(i)}$. If there exists a perfect matching $\sigma: [n] \rightarrow [n]$ in G , then σ will produce a term with coefficient ± 1 in $\det E(G)$, which will not get canceled out by any other term. Similarly if $\det E(G) \neq 0$, then a nonzero term gives a perfect matching $\sigma: [n] \rightarrow [n]$. $\square$

We could test $\det E(G) \neq 0$ via polynomial testing and Lemma 11.2.